

StockAgent

Technical design & knowledge transfer

Understand the three loops, IPO evidence, portfolio decisions, profit and loss, scheduled work, and the changes still ahead.

Edition 2026-09-26

Code-grounded explanations for engineering and product teammates.

Human testing duties are a separate document.

Source: docs/TECHNICAL_DESIGN.md · SHA-256 2dcf96cc94c5

Inside this guide

1. Reading the evidence
 2. Product map and three loops
 3. Runtime, configuration and storage
 4. Research: input to recommendation
 5. Learning: forecast, review and memory
 6. Portfolio: advice and virtual execution
 7. Profit/loss and the different marksheets
 8. Discovery and IPO functionality
 9. Scheduled jobs and event hooks
 10. Delivery, interface, identity and operations
 11. Changes already present and planned redesign
 12. PI changes included now
 13. Validation and KT sequence
-

Edition: 2026-09-26 · **Audience:** engineers and teammates learning the product

Code inspected: 4c4728ae5b879e26430ed0f58dee2728cc126cd9

First edition 2026-09-15 at 9a805878; maintained per story since. The revision above is the one the whole body describes. `check_kt_docs.py` fails if a linked source file or a documented job ID is absent at that revision.

PDF: StockAgent-Three-Loops.pdf, generated from this Markdown.

This is the current KT source, replacing the May technical reference. It explains the implementation and the changes planned in PI-2026-09. The separate human testing guide divides practical testing duties; it is not an implemented human-approval workflow.

1. Reading the evidence

Label	Meaning in this edition
Current code	Traced in this checkout. Flags, inputs and runtime data determine whether a path actually runs.
Locally checked	Existing tests run in an isolated copy. Exact results and limits are in the validation receipt.
Production observation	Dated evidence: the September 10 audit, section 10's 2026-09-21 email diagnosis, a September 15 deployment SUCCESS at 9a805878, the 2026-09-23 read-only log inspection of learned weights, and the 2026-09-24 SA-039 weight baseline. Those logs came from deploy d9c459ae (commit e8df088). Deploys carrying the header revision's code were inspected read-only on 2026-09-25 (7ebd06c5: the 16:30 review ran in adapt) and on 2026-09-26 (da9df6cf: all 20 tickers in observe, per the activation record).
PI target	Intended behavior, not completed functionality. SA-039 was accepted by its fresh review on 2026-09-25, and production has run observe since 2026-09-26 (activation record; verification after the next reviews is pending). Every other story from SA-001 to SA-047 is todo. Three are stretch.

The September audit records unresolved label, timing, health, weight-bound and operational defects. Adaptive state exists; improved prediction performance has not been demonstrated. No production job, backfill or notification was triggered for this KT.

Vocabulary

Term	Plain-English meaning
Ticker / instrument	A market symbol / the actual security it represents. A renamed or reorganized company is not automatically the same investment.
Sector / graph sector	The company's business sector / the analysis implementation selected for it. A pharma stock can use a generic graph while retaining pharma identity.
LLM / agent / dimension	Language model / component with an analysis task / one score such as fundamentals. Multiple dimension names do not establish independent evidence.
Composite / weight	Combined score / importance assigned to a dimension.

Term	Plain-English meaning
Envelope	Stored forecasts for future trading sessions, including prices, verdicts and uncertainty information.
Feedback / lesson / dossier	One review result / reusable knowledge / longer-lived company observations and questions.
Advice / transaction	A portfolio recommendation / a recorded virtual buy or sell. They are separate records.
Cohort / matured outcome	A defined group of predictions / an outcome whose observation period has finished.
Idempotent	Repeating the same operation does not apply its effect twice.
Cache / projection	A reusable fetched result / a secondary representation of another store. Neither guarantees freshness or authority.
PI	Improvement plan; accepted story statuses are separate from features already present in the baseline.

2. Product map and three loops

StockAgent combines listed-equity research, adaptive feedback, a virtual portfolio, discovery, IPO information and reports. Three connected product loops explain the main flow:

1. **Research:** collect evidence, score dimensions, combine them and produce a report.
2. **Learning:** compare forecasts with observations, record feedback, adjust weights and maintain lessons and company knowledge.
3. **Portfolio:** evaluate held positions, record advice, optionally execute it in a virtual account, then produce transactions, value history and summaries.

The August **Three Loops PI** named an improvement program covering routing, data health and grading. Its proposed redesigns are not all implemented.



The nightly outcome auditor measures issued advice. The watchdog checks operational milestones. Neither is the daily feedback agent. There is no general human-approval queue between these components.

Repository orientation

Location	Responsibility
API server and routes	FastAPI startup, API and streaming interfaces.
src/backend/shared	Shared orchestration, canonical schemas, prompts and settings.
src/backend/sectors	Native implementations, generic graph and sector registry.
services/data	Fetchers, context assembly, caches, durable stores and backup.
core/intelligence	Forecasting, feedback, regimes, seasonal logic and company knowledge.
core/portfolio, core/discovery, core/ipo	Virtual money path, listed-stock discovery and IPO evidence.
core/audit, core/ops/watchdog, core/delivery	Outcome measurement, operational monitoring and delivery.
src/frontend/prototypes	Served React/JSX application, PWA and data adapters.
tests	Unit, integration and contract checks; their presence alone is not acceptance.

Some paths are compatibility re-exports. For example, `core/config/settings/base.py` forwards to `src/backend/shared/config/settings/base.py`. Follow the import before editing; two import paths need not mean two implementations.

3. Runtime, configuration and storage

Runtime

Dockerfile uses Python 3.11 and runs `services.api.server:app` with two Uvicorn workers. It copies `src/backend/` to `/app/backend/`; local tests add `src` to the import path. The served frontend is the prototype JSX application. `package.json` provides development tooling; there is no compiled app frontend build step. The checked-in C# scheduler is not launched by this Dockerfile.

Both workers run `startup`. One binds a localhost TCP port, default 59321, and becomes the background owner. It starts RL self-heal, APScheduler, cleanup and the optional Atlas outbox drainer; the other serves API requests. This guard is container-local, not a distributed lock or continuous standby takeover. `GET /health` returns process-level ok; it does not verify background ownership, storage, jobs, data or delivery. SA-029 addresses that gap.

Configuration

The loader resolves a declared environment override, then `config.yaml`, then a code fallback. Not every YAML field has an environment override. Most settings are resolved at import, so editing a file does not imply live reconfiguration.

Checked-in setting	Meaning
<code>scheduler.enabled: false</code>	Local startup default; production overrides are separate.
<code>scheduler.feedback_cron: "30 16 * * mon-fri"</code>	Daily review default, interpreted in Asia/Kolkata.
<code>rl.hard_bind_verdict_enabled: true</code>	Research category follows the composite; model <code>final_score</code> remains separate.
<code>rl.learning_mode: adapt</code>	SA-039 switch, env <code>RL_LEARNING_MODE</code> . <code>adapt</code> keeps learned weights and lesson emphasis live; <code>observe</code> contains them (section 5). Any other value fails closed to <code>observe</code> .
<code>rl.control_lane_enabled,</code> <code>rl.scorecard_enabled: true</code>	Control/evaluation paths configured; not proof of valid prospective comparison.
<code>ipo.enabled: true; ipo.gmp_enabled: false</code>	IPO refresh and grey-market-price fetching have separate gates.
<code>delivery.enabled: true;</code> <code>delivery.email_enabled: false;</code> <code>delivery.push_enabled: true</code>	Job scheduling, transport enablement and credentials are separate conditions.
<code>watchdog.enabled: true;</code> <code>watchdog.prep_enabled: true</code>	Monitoring can perform configured prep as well as report results.
<code>audit.enabled: true</code>	Nightly outcome grading configured.

These are repository values, not a fresh inventory of production variables. Credentials and `.env` contents do not belong in KT or testing evidence.

Persistent records

Records	Writer / consumers and scope
data/predictions/<sector>/<ticker>/	PredictionStore: envelopes, feedback, controls, weight memory, lessons and dossier, plus <TICKER>_weight_observations.json (SA-039 observe-mode proposals). RL writes; advisor, evaluation and UI read. SA-009/SA-017 still need to reconcile historical aliases and mixed stores.
_shared_ledger.json per sector; _market_ledger.json at prediction root	Shared lessons can affect more than one ticker; bad attribution can propagate.
data/portfolio/<user>/	PortfolioStore: holdings/cash, advice and transaction JSONL, value history, dated digests, briefs and weekly reviews.
data/ipo/	Historical issue facts and captured demand snapshots; reports derive from these records.
data/market_cache/ipo.json and EOD parquet	Cached IPO information and exchange price history; date, source and stale state matter.
SQLite score, log, user and Atlas stores	Historical scores, telemetry, sessions and relational indexes/outbox. Secondary tables may lag primary records.
data/scheduler_job_outcomes.json; data/watchdog_state.json	Operational results and milestone state, not proof of financial performance.
data/backups/	Local rotation, distinct from demonstrated off-site recovery.

PortfolioStore's JSONL ledger is the inspected advice source. September 10 found incomplete/stale optional Atlas projections. An empty projection does not prove no advice was issued.

4. Research: input to recommendation

Trace: registry → base orchestrator → bundle builder → unified analyst → aggregator.

1. Resolve input to a company, symbol and sector. Known managed symbols can bypass model-based resolution.
2. Select the graph. Automobile, banking/BFSI, IT and renewable energy have native implementations; other sectors use generic analysis. The RL router delegates graph selection to the central registry; August A1 is present.
3. Assemble company/sector news, fundamentals, technicals, macro, flows, peers and company knowledge. NSE prefetch data is shared. Sections may be cached, unavailable or not applicable.
4. The preferred unified analyst scores the sector dimensions together. The legacy per-dimension pool remains a fallback. A whole research request can involve resolution, aggregation, retries and extra provider calls; it is not necessarily one LLM call.
5. The aggregator combines weighted scores, excludes errored contributions and normalizes surviving weights. With no surviving weight it falls back to a neutral composite. An LLM produces narrative and report fields.

6. With hard-bind enabled, configured score bands determine the categorical verdict. The raw model verdict is logged for comparison; its numeric `final_score` remains separate.
7. Score history, run summaries, logs and data-health records support the user report and later diagnosis.

Example: scores 0.8 and 0.4 at weights 0.75 and 0.25 give composite $0.8 \times 0.75 + 0.4 \times 0.25 = 0.70$. The configured band containing 0.70 gives the bound verdict. A different LLM `final_score` is a different field, not necessarily an arithmetic error.

Current gap: durable health exists, but empty/failed data and downstream eligibility do not form a reliable gate. A surviving subset can still produce an actionable report. SA-002 defines usable health; SA-003 gates action and learning; SA-008 handles unresolved instruments; SA-010 corrects benchmark arguments. SA-025 measures calls before SA-026 consolidates sector definitions and SA-027 retires only justified fallback duplication.

Planned redesign (adopted 2026-09-26, not implemented). The one-engine design retires the per-sector graphs. Today all five graphs already run one bundle and one reasoning-model call; the LangGraph pool is only a fallback. They differ mainly in five scoring schemes (37 dimension slots under 24 names), and the LLM re-scores numbers the code has already computed. Generic-graph stocks are valued against automobile peers, and missing data is scored 0.5. The plan has four parts: - sector knowledge becomes a lens YAML (peers, benchmark, KPIs, news terms and macro drivers), resolved from NSE's industry field; - code computes five factors (Value, Quality, Growth, Momentum and Risk) as percentiles against real peers and the stock's own history; - one LLM reader returns dated, sourced events, and code turns them into a sixth factor, Catalyst; - code combines the six at equal weights and maps the result to the existing bands. With fewer than 4 of 6 factors it gives no directional verdict. The LLM explains the verdict but cannot change it.

SA-044 fixes the peers first. SA-026, SA-045 and SA-046 run in shadow. SA-047 switches only when the engine is right on at least 50% of disagreements with today's analyst, and SA-027 then deletes the sector packages.

5. Learning: forecast, review and memory

Trace: forecast generation, interpolator, daily review, feedback agent, weight adapter, prediction store, learning mode.

Forecast generation

The workflow loads weights and knowledge, runs analysis and creates an envelope across future exchange sessions. A model-shaped profile, static fallback and simulated paths supply forecast prices and bands. Generated forecast rows are not independent observed samples or guaranteed probabilities.

The monthly trigger is the **calendar first at 09:00 IST**. The generated rows use exchange trading dates. Do not call that trigger the first trading day. Shocks, thesis changes and later evidence can revise/archive envelopes. SA-013 will define immutable issuance identity so revised knowledge does not replace the earlier decision's evidence.

Daily review walkthrough

1. The scheduler selects the previous exchange trading session.
2. Daily review finds its envelope row and obtains the actual close, including the configured verification path.
3. It computes price error and the current direction label. Some small, correct misses skip a fresh analyst run.

4. Otherwise it may rerun analysis. With hard-bind enabled, current code can grade the old outcome against the fresh report's verdict.
5. Feedback classifies misses and attributes them to dimensions; weights, lessons and other learning state are updated.
6. Conditional thesis review, reforecasting, dossier curation and shared knowledge updates can run under their feature gates.
7. After harvesting reviews, the scheduler invokes the portfolio pipeline. Returning successfully does not establish that every ticker got feedback.

Grading defect: an example everyone can follow

Suppose the issue-time close was 100, the predicted close 110 and the observed close 105. The stock rose **5%**, but undershot its forecast by **4.55%**. Current `classify_direction(actual, predicted)` compares against the forecast and labels this DOWN. That measures error direction, not investment direction. A new verdict computed after observing the outcome can also become the graded verdict. Label definition and issue-time information are separate problems.

Existing tests can describe those code paths without establishing the desired financial contract. SA-012 defines the target; SA-013/SA-014 freeze and grade information available when the decision was issued.

What adaptation means here

Weights, accuracy history, lessons and dossier records change over time. Regime and lesson modifiers affect research inputs. This is an adaptive feedback system; it does not demonstrate that adaptation beats a fixed policy. The audit also reproduced final weight-bound violations and repeated ensemble trend counts presented under different agent names.

Production adapts live weights on every eligible review. On 2026-09-23, production logged `technical` at 0.0 for 5 of the 6 tickers that have that agent. The 2026-09-24 baseline reproduced it: the defaults are 0.12 in the generic graph and 0.10 in the renewable graph. Counting `pattern_analysis`, the chart agent of the other graphs, 9 of 18 tickers had a chart weight of 0.0 and 16 of 18 were below half their default. All 19 reviews that day wrote a new weight version.

SA-039 (accepted 2026-09-25; production observe since 2026-09-26): `r1.learning_mode` has two values. `adapt` is the checked-in value and the behaviour described above. `observe` contains learning:

- Forecasts, re-forecasts, public analysis, and the review's re-scoring and forecast revision aggregate with the sector's configured default table (the table a ticker with no learning state gets), whatever the file stores.
- The review never writes weight memory. The adapter runs on a copy, and its would-be weights, deltas and reason go to `<TICKER>_weight_observations.json`, one record per review date.
- Tagged-lesson emphasis and the older category micro-adjustment stop moving agent scores. Lessons are still recorded, and `claims_fired` stays empty because no claim acted.

Example: a ticker stores `technical = 0.0` at v41. In `observe` mode its forecast uses 0.12; after the review the file still says 0.0 at v41, and the observation record shows what v42 would have been. Switching back to `adapt` resumes the stored weights unchanged. An unrecognised value fails closed to `observe` with a warning. The mode is shown in the review's start and completion log lines, on each envelope (`learning_mode`), and by `/ui/r1/weights/{ticker}` (with the live `decision_weights`) and `/ui/r1/summary/{ticker}`. The paper lane still never trains or writes weights, and the absurd-price-error guard still skips the adapter. Like every other decision path, the paper lane uses the sector defaults in `observe`. SA-039 does not contain regime multipliers, thesis multipliers,

the conviction streak, miss-counter prompt enhancements or dossier text. **Production has run observe since 2026-09-26 at about 06:40 IST.** The owner set the Railway variable `RL_LEARNING_MODE=observe`, which overrides the checked-in `adapt` (activation record). Rollback is deleting the variable. Forecast rows issued before activation keep their verdicts and closes; the review re-weights only their confidence. The first fully contained cycle is therefore the next monthly forecast (the 1st, 09:00 IST).

Leaving observe (planned, not implemented). In the current code, `observe` re-proposes one step from the frozen stored file each night, so its records do not accumulate into a learner. SA-043 plans a shadow learner. It learns six factor weights pooled across all stocks (one-engine design), starting from the defaults and learning on the corrected target (SA-012 to SA-015). For each issued decision, it records the default verdict and the shadow verdict from the same factor scores. SA-022 then scores only the decisions where the two disagree. Learning may return only when all of these hold: - at least 100 effective disagreements, over at least 3 months and 3 lenses; - the shadow is right on at least 60% of them; - its wins are consistent month to month and robust to dropping its best lens; - no factor's weight has collapsed to its bound, or below half its default, for 4 weeks.

The pass must hold at two consecutive monthly looks, and the owner still decides. `adapt` would then resume from the shadow's weights, never from the stored pre-fix weights. A reverse tally in the first month (below 45%) returns learning to `observe`.

SA-015 requires retry-safe updates and final bounds, SA-016 fixes attribution, and SA-017 records comparable history. SA-020/SA-021 address overlap, calibration and recovery evidence. SA-022 requires prospective comparisons against a frozen policy; SA-023 tests lessons on probation. Future matured market observations remain necessary even after implementation acceptance.

Event ingestion adds company observations; the curator maintains knowledge and questions; the research job attempts answers. These are knowledge tasks, not trades. Discovery paper tracking uses a separate store root with learning writes disabled for that lane; it is not live investor performance.

6. Portfolio: advice and virtual execution

Trace: pipeline → advisor → executor → store → digest.

The pipeline loads active users' portfolios, obtains closes and signals, evaluates held positions, records advice and optionally executes it. The LLM narrates the rule-driven decision; triggers remain available when narration fails.

Research verdict	Portfolio action
STRONG BUY, BUY, NEUTRAL, SELL, STRONG SELL	ADD, HOLD, TRIM, EXIT, SWITCH
Assessment of a stock at analysis time	Action for an existing holding, using entry cost, risk, position size and evidence

A BUY report does not automatically purchase shares. A portfolio HOLD does not mean a flat forecast.

Advisor precedence

The cascade is **EXIT** → **TRIM** → **ADD** → **HOLD**. EXIT includes stop breach, bearish thesis/shock conditions, crisis conditions and an armed trailing stop. TRIM considers profit with declining confidence/reversion. ADD requires bullish, sufficiently healthy conditions and position headroom. EXIT may become SWITCH when an eligible replacement exists. A holding-age rule can soften certain TRIM actions to HOLD; it does not override EXIT. These are product rules, not individualized tax or investment advice.

Execution and persistence

The global gate, user's virtual-autopilot choice and cash-accounting state all matter. Execution reloads the portfolio under a lock, sells before buying, and uses transaction IDs plus a monotonic review-date marker. Future and already-completed dates are rejected/skipped. Cash, sizing and eligibility can prevent a recommended action from becoming a transaction.

Transactions are appended before saving holdings/cash. A crash in between can leave the ledger ahead of the portfolio; deduplication is not an atomic multi-file transaction. Reconciliation handles this gap. Corporate actions affect adjusted quantities and cost. No broker-order execution path was found.

7. Profit/loss and the different marksheets

"Marksheet" covers several distinct reports. Each needs its own question, dates, denominator and exclusions.

Virtual P/L

portfolio_api.py exposes holdings, transactions, advice, digest and /portfolio/performance.

Measure	Meaning
Holding market value	Adjusted quantity × marked price.
Unrealized holding P/L	Adjusted quantity × (marked price – adjusted average cost).
Realized P/L	Ledger gain/loss on quantities actually sold virtually.
Total equity	Remaining holdings' market value + deployable cash.
Total return	(total equity – capital_in) / capital_in, when required values exist.
As-of date	Date of the valuation, potentially different from page-open time.

Example without fees/corporate actions: start with 2,000 and buy ten units at 100. Cash is 1,000. At 110 the holding is 1,100, equity 2,100 and unrealized P/L 100. Sell four at 110: cash becomes 1,440, remaining holding value 660, realized P/L 40 and remaining unrealized P/L 60. Equity stays 2,100.

Current performance reads at most 2,000 transactions and 400 history rows. With no history, a failed price lookup may fall back to adjusted purchase cost. The digest sums priced holdings only, and its portfolio_value excludes cash. These scope differences need interpretation; a displayed total is not proof all prices are fresh or all history is included. These virtual results do not establish executable net returns after fees, spread, tax and slippage.

Report families

Report	Present meaning	Limitation / PI work
RL Monitor / analytics	Envelope, feedback, miss and weight records.	Inherits label/timing/history problems; SA-012–SA-017, SA-024.
Weekly scoreboard	28-calendar-day lookback, non-HOLD calls at least five calendar days old, latest supplied closes.	Not fixed matured horizons; current-holding price selection can omit exited stocks. SA-018/SA-019.
Monthly scorecard	Agent/control/simple-baseline lanes and previous-month comparisons.	Completeness, timing and denominators need repair. SA-018.
Learning evidence	Retrospective adapted/base/uniform replay and lesson associations.	Not a prospective controlled experiment of the whole system. SA-020–SA-023.
Nightly advice auditor	Issued advice, alert, shelf, switch and IPO lanes; default 10/30/60 trading-session horizons, and 1/5/21/63/126/252 from listing on the IPO lane.	Separate from weekly heuristics and actual account P/L. IPO rows are excluded from the rendered report.
IPO history report	Subscription groups and issue-price returns at available horizons.	Descriptive associations with sample/feature gaps, not a validated prediction model.

Audit rules judge HOLD/ADD correct when excess return over the benchmark is nonnegative, and TRIM/EXIT/SWITCH correct when negative. The switch-pair lane separately asks whether destination beat origin over the same dates; a tie does not establish benefit. An EXIT can score correctly while the actual virtual trade realizes a loss: these measure different things.

The IPO lane (grade_ipo_lane) grades the P3 IPO verdicts under `is_ipo_correct`, a separate rule from the advice vocabulary. Its entry price is the **issue price**, not a market close, and its horizons count trading days from listing with horizon 1 being the listing day itself. Only one row per issue can carry a True/False: the listing-day horizon, and only when the verdict was taken after the book closed and its lean asserted a direction. Every other horizon is recorded with `correct` unset, so it enters no hit-rate. These rows are written to the same store and excluded from the rendered audit report: the P3 model remains dark until its visibility gate is met, and a displayed hit-rate would be that surface. No production IPO row has been graded yet.

Example: stock +5%, benchmark +8% means excess **–3 percentage points**. HOLD is incorrect under that relative-return rule despite a positive stock return. Every marksheet should identify its rule, horizon, matured sample count, missing data and excluded cases.

8. Discovery and IPO functionality

Discovery

`run_discovery_cycle` connects EOD sync, bulk/block activity, optional recent-IPO candidates, screen, deep dives, conviction shelf and paper reviews. Failures can degrade individual stages. The Saturday job and manual discovery endpoint use this entry point. A shelf candidate, watchlist promotion and virtual purchase are separate steps.

IPO layers

Layer	Current source and output
Calendar, offer and bids	ipo.py, ipo_offer.py, ipo_bids.py: normalize and enrich issues. Failed refresh preserves previous cache with degraded/stale information.
Captured snapshots	signals.py: observed refresh facts with hour/content deduplication. velocity.py: demand changes derived on read.
Historical evidence	history.py, outcomes.py, report.py: facts, realized curves and subscription-bucket summaries.
P3 model and deep dive (dark)	research.py, extract.py: browsed, corroborated Substance facts with source URLs. hype.py, substance.py, verdict.py: deterministic indices and the §3 verdict grid over captured and browsed facts. deep_dive.py: the 19:00 ipo_deep_dive sweep (T-1 research run, post-close re-read from cache) that writes to the append-only store in verdicts.py. narrate.py: a sourced research note stored on that row, written by the bulk model from the SAME structured findings the indices read — the verdict is never passed to it, every number in the prose must appear in the findings, an advice/verdict vocabulary rejects it, and a rejected or failed note falls back to a deterministic template. Writes only: no verdict, index or quadrant reaches any surface until the ipo_verdicts_visible_gate milestone is judged on forward rows.
Forward grading of the P3 verdict (dark)	listing.py resolves the listing date and issue price from the P1 spine, then the NSE cache; grade_ipo_lane grades the newest stored verdict per issue against the tape at 1/5/21/63/126/252 trading days from listing, entry price = issue price. Only the listing-day row of a post-close verdict with a directional lean can be scored; the rest carry the return and no claim. Rows land in the existing per-user audit store and are excluded from the rendered audit report . No production row exists yet: the job reaches production only on deploy, and grading also waits on the issue reaching the P1 spine, which scripts/ipo_backfill.py still rebuilds manually.
Recent-listing ranking	ipo_tracker.py: listing evidence, delivery trend, bulk accumulation and optional subscription score discovery candidates.
User surfaces	IPO-watch in briefs, weekly/discovery context and shelf. The brief's demand lean (_ipo_lean) judges subscription against size-tiered bands from delivery.brief_ipo_size_tiers; an issue whose size was not read uses the scale-free scalar thresholds, never the largest tier. The lean is a labelled heuristic, not the P3 verdict. Inspected routes do not provide a dedicated /ipo/predict API or complete standalone IPO prediction page.

History uses **1, 5, 21, 63, 126 and 252 trading sessions**, measured from **issue price**. Unmatured horizons are absent, not zero. Benchmark excess subtracts the listing-to-horizon benchmark return; that window does not capture the stock's issue-to-listing move in the same way. Do not interpret the result as a fully matched investable strategy. Missing subscription is excluded, and report bucket means below a sample count of ten are suppressed.

The recent-listing tracker is a separate heuristic. It needs at least five EQ sessions and configured price/liquidity guards. Component weights are 40% listing evidence, 20% delivery trend, 15% bulk accumulation and 25% subscription, renormalized over available components. It can fall back to first cached close when issue price is absent; that differs from historical-outcome calculation.

GMP fetching exists behind a separate disabled-by-default gate. Missing GMP does not mean zero. Lock-in alert dates use fixed 30/90/180-calendar-day offsets from listing; these are software heuristics, not verification of an individual issue's contractual dates.

The historical **Prospect** design planned further modeling after P0/P1/P2 collection. Current code has collection, history, captured signals, heuristic candidate ranking and, since PI Prospect Sprints 2–4, the dark P3 model, deep-dive job, narrator and forward-grading lane in the table above. None of it reaches a user, and no production

verdict row has been graded. A validated IPO application/allotment or listing-gain predictor is therefore still not established, and is not silently added to the September remediation scope.

9. Scheduled jobs and event hooks

Source: scheduler.py. These are in-process APScheduler jobs, not separate OS cron jobs. There are **24 possible job IDs**, including three IPO jobs. Registration depends on gates and valid cron configuration; registration alone is not execution. All times are **Asia/Kolkata (IST)** defaults.

Job ID	Schedule	Work / condition
prompt_daily_deploy	Daily 00:00	Prompt publishing; handler/configuration can write externally.
ops_watchdog	Daily 06:30	Milestones/checks/configured prep; watchdog gate.
macro_daily_news	Mon–Fri 07:30	Policy/RBI feed; macro-news gate.
ipo_refresh_am	Daily 08:00	Calendar/bids/snapshots; IPO gate.
preopen_shock_check	Mon–Fri 08:45	Shock/reforecast; feature/calendar checks in handler.
morning_brief	Mon–Fri 08:50	Brief assembly/delivery; delivery gate.
macro_market_news	Mon–Fri 09:00, 12:00, 15:00	Intraday macro feed; macro-news gate.
rl_daily_review	Mon–Fri 16:30	Previous trading session; configured feedback cron.
ipo_refresh_pm	Daily 17:45	Evening IPO update before Sunday weekly review; IPO gate.
ipo_deep_dive	Daily 19:00	T–1 IPO research + post-close re-read into the dark verdict store; IPO gate.
bhavcopy_daily_sync	Mon–Fri 19:00	Official EOD cache; discovery gate.
atlas_universe_recompute	Daily 23:00	Demand/cadence; no-op if Atlas disabled.
atlas_cost_rollup	Daily 23:15	Cost aggregation; Atlas gate in handler.
atlas_retention	Daily 23:20	Retention; Atlas gate in handler.
data_backup_nightly	Daily 23:30	Local archive/rotation and configured off-site attempt.
audit_nightly	Daily 23:45	Matured outcome grading/breach reports; audit gate.
ledger_cleanup_weekly	Monday 03:30	Stale-lesson cleanup.
event_ingest_weekly	Saturday 10:00	Dossier events; event-ingest gate.
research_loop_weekly	Saturday 11:00	Open company questions; research-loop gate.
discovery_weekly	Saturday 12:30	Screen/deep dives/shelf/paper; discovery gate.

Job ID	Schedule	Work / condition
weekly_review	Sunday 18:00	Portfolio summary/index watch; delivery gate.
scorecard_monthly	Calendar first 02:00	Previous-month scorecard/evidence; scorecard gate.
r1_monthly_forecast	Calendar first 09:00	Forecast envelopes.
r1_calendar_update	December 31 23:00	Holiday calendar refresh.

Startup self-heal, the post-review portfolio/digest hook and the outbox drainer are **not additional cron jobs**. An 08:50 brief is not guaranteed to wait for an unfinished 08:45 shock check. Close clock times are not dependencies.

Current review success counts can include returned failure/skip statuses, such as no envelope. A pipeline flag can mean the call returned, rather than every holding completed. HTTP 202 means accepted for background work, not finished. SA-004 repairs outcomes, SA-011 operational backlog and SA-029 readiness/ownership recovery.

10. Delivery, interface, identity and operations

Briefs, weekly reviews and digests persist independently of transport. Channels send email and web push. Email has two transports: SMTP (STARTTLS) and the HTTPS Resend API. `EMAIL_TRANSPORT` in settings defaults to `auto`, which uses Resend only when `RESEND_API_KEY` is set and SMTP otherwise; `resend` or `smtp` forces one. The repository's `delivery.email_enabled` is `false`; the `DELIVERY_EMAIL_ENABLED` environment variable overrides it.

`outbox.py` provides retry/dead-letter handling when its Atlas path is active (3 attempts, 1/5/30-minute backoff in `config.yaml`). Each email row's recipient comes from `resolve_recipient()`: the owning account's address in `users.db`, else `DELIVERY_EMAIL_TO`. A failed send stores its reason in `outbox.last_error`, which is cleared when a retry succeeds; an additive migration in `atlas_store.py` adds the column to existing databases. Stored, queued, provider-accepted, received and read are separate states. `last_error` explains a failure; an empty one does not prove receipt.

Dated production observation. September 10 recorded email failures and no confirmed app-created off-site backup copy. The email cause was identified on 2026-09-21: Railway disables outbound SMTP on its Hobby plan, and production had logged `[Errno 101] Network is unreachable` on every send since 2026-07-16. After a plan upgrade and redeploy, one triggered brief reached the inbox that day. That is a single observed delivery, not a measured delivery rate. Whether production sets `RESEND_API_KEY` was not inspected, and delivery was not remeasured for this edition.

Known gaps in the shipped delivery code (`590bc9f`, which partially satisfies SA-006 without accepting it):

- `resolve_recipient()` also falls back to `DELIVERY_EMAIL_TO` when the `users.db` lookup raises. A transient failure can therefore route a beta tester's brief to the owner's inbox.
- Resend's default shared sender (`RESEND_FROM`) delivers only to the Resend account owner's address. Per-account delivery needs a verified-domain sender.
- Two outbox tests pass only when an ambient `DELIVERY_EMAIL_TO` is present. This is routed to SA-005.

SA-006/SA-007 address the remaining delivery and backup acceptance gaps.

The frontend uses React JSX, runtime browser transformation and PWA assets. Chat uses a streaming tool loop with potentially paid provider calls. Prompt editing can feed scheduled publishing. These are not inert testing screens. Bearer sessions, roles and machine-key authentication are implemented; portfolio identity comes from the authenticated user. Some intelligence read routes remain public. The RL client has demo/fallback paths on API failure, so populated charts alone do not prove live data. SA-001 repairs unsafe Markdown rendering; SA-024/SA-032 address evidence and public/demo policy.

Durable logs connect run IDs, tickers, warnings, LLM usage and health. Bundle call counters are not complete nested-provider/fallback cost accounting; SA-025 addresses this. The watchdog reads milestones.yaml, runs checks, persists state and may perform configured preparation. A sent milestone notice is not milestone completion. Local backup files and healthy HTTP responses do not prove recovery or successful jobs.

11. Changes already present and planned redesign

Topic	Current implementation	Still planned / unverified
Sector routing	Shared graph selection via registry.	Complete store lineage, and sector lenses resolved from NSE's industry field (one-engine design; SA-026).
Analysiss	Unified scoring plus surviving legacy fallback.	Actual call accounting. A factor engine (computed factors, one text reader, code decides, LLM explains) proven in shadow, then the graphs and fallback retired (SA-044–SA-047, SA-027).
Data health	Durable health/run records.	Usable-data semantics and recommendation/learning gates.
Verdict binding	Deterministic category enabled in YAML, raw model verdict logged.	Correct issue-time grading and final adaptive constraints.
Portfolio	Per-user advice/execution, stops, switches and ledgers.	Stronger upstream evidence and report reconciliation.
IPO	Calendar, history, snapshots, recent-listing screening, size-tiered brief lean, and the dark P3 model, deep dive, narrator and forward-grading lane (section 8).	Forward evidence for P3 and its ipo_verdicts_visible_gate; no verdict reaches a user; outside default September scope.
Operations	TCP singleton, outcomes, watchdog, outbox with last_error, per-account recipients, SMTP/Resend transports and backup code.	Truthful counts, proven recovery, measured delivery, the recipient-fallback gap and readiness. The observability design adds planned job-run and source-health ledgers, post-job checks, a read-only status fetcher and an outside witness (SA-034–SA-036, SA-040–SA-042).
Frontend	JSX/PWA with live adapters and some fallback/demo paths.	Sanitization, honest unavailable states and optional build cleanup.

Historical specifications retain what was intended at the time. The September audit supersedes unsupported August claims about correct learning, automatic security succession or unused fallback cost. Legacy code should only be retired with measured replacement coverage.

12. PI changes included now

The table below includes the planned destination now. **SA-039 is accepted (production observe since 2026-09-26); every other SA story is todo.** Accepted state/dependencies are in STATE.json. DOC-001 is this user-requested documentation refresh; it does not close SA-031 or any upstream remediation.

Story	Planned change	KT sections	Dependencies
SA-001	Sanitize every chat Markdown rendering boundary	10 Interface	None
SA-002	Make data-health records describe usable evidence	4 Research	None
SA-003	Gate recommendations and learning on essential data	4 Research; 5 Learning; 6 Portfolio	SA-002
SA-004	Count daily-review outcomes truthfully	9 Jobs	None
SA-005	Establish a clean, isolated test and CI baseline	13 Validation	None
SA-006	Repair delivery transport and expose dead letters	10 Delivery	None
SA-007	Make backups independently recoverable	3 Storage; 10 Recovery	None
SA-008	Quarantine unresolved securities with explicit lifecycle records	4 Identity; 8 Discovery	SA-003
SA-009	Inventory and reconcile prediction-store ownership	3 Storage	None
SA-010	Pass sector benchmarks through technical-data calls	4 Research	None
SA-011	Turn durable errors into a deterministic operations backlog	9 Jobs; 10 Operations	SA-002, SA-004
SA-012	Specify and test the learning target before changing it	5 Learning; 7 Marksheets	SA-005
SA-013	Persist immutable, genuinely prospective forecast identities	3 Storage; 5 Learning	SA-012
SA-014	Grade frozen decisions and issue controls before their outcomes	5 Learning; 7 Marksheets	SA-003, SA-013
SA-015	Make adaptive updates idempotent and preserve final weight bounds	5 Learning	SA-014
SA-016	Replace blame-based shared credit with identifiable outcomes	5 Learning	SA-015
SA-017	Attach historical lineage and quarantine incomparable feedback	3 Storage; 5 Learning	SA-009, SA-013

Story	Planned change	KT sections	Dependencies
SA-018	Build complete, matched monthly evaluation cohorts	7 Marksheets	SA-014, SA-017
SA-019	Evaluate suggestions across weekly and fortnightly cohorts	7 Marksheets	SA-018
SA-020	Compute effective samples from trading-session overlap	5 Learning; 7 Marksheets	SA-013
SA-021	Validate calibration and weight recovery with minimum evidence	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020
SA-022	Run prospective adapted and frozen-policy experiments	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020, SA-043, SA-047
SA-023	Put learned lessons on measurable probation	5 Learning	SA-022
SA-024	Show verifiable learning evidence and honest unavailable states	7 Marksheets; 10 Interface	SA-019, SA-020, SA-022
SA-025	Count actual provider calls and fallback cost	4 Research; 10 Cost	SA-011
SA-026	Consolidate sector definitions into sector lenses	2 Modules; 4 Research	SA-003, SA-009, SA-010
SA-027	Retire redundant fallback only when measured replacements work	4 Research; 11 Changes	SA-025, SA-026, SA-047
SA-028 (stretch)	Repair packaging and compile the browser client	3 Runtime; 10 Interface	SA-001, SA-005
SA-029	Verify readiness and recover background ownership	3 Runtime; 9 Jobs	SA-004, SA-007
SA-030 (stretch)	Reconcile or retire stale Atlas projections	3 Storage	SA-009, SA-013
SA-031	Verify final architecture and operator-documentation consistency	All: final consistency check	SA-024, SA-027, SA-029
SA-032 (stretch)	Define and enforce the public-read and demo-data policy	10 Interface	SA-001, SA-024
SA-033	Lock reproducible runtime dependencies	3 Runtime; 13 Validation	None
SA-034	Detect silent degradation in the watchdog	9 Jobs; 10 Operations	SA-036, SA-040
SA-035	Surface LLM and search provider credit exhaustion	10 Operations; 10 Cost	None
SA-036	Record durable outcomes for every critical job	9 Jobs	None
SA-037	Keep deploys from dropping scheduled jobs	3 Runtime; 9 Jobs	SA-036
SA-038	Judge and retire lapsed production-verification milestones	9 Jobs; 10 Operations	None

Story	Planned change	KT sections	Dependencies
SA-039	Contain adaptive learning: observe-only, live weights back to defaults	5 Learning	None
SA-040	Record per-source fetch health and detect new error types	9 Jobs; 10 Operations	None
SA-041	Provide a read-only production status fetcher	10 Operations; 13 Validation	SA-036, SA-040
SA-042	Add an outside witness for missed jobs (Healthchecks.io → Telegram)	9 Jobs; 10 Operations	SA-036
SA-043	Run a shadow learner and record paired decisions in observe mode	5 Learning	SA-015, SA-039, SA-045
SA-044	Compare valuations only with same-industry peers	4 Research	None
SA-045	Compute the five universal factors deterministically	4 Research	SA-026
SA-046	Read text into dated events and a Catalyst factor	4 Research	SA-026
SA-047	Decide with the factor engine; switch when it is not worse	4 Research; 5 Learning	SA-045, SA-046, SA-014, SA-020

Maintain documentation with each story

Each implementation updates the affected current-code section, identifies which target behavior has landed, adjusts relevant human test cases and regenerates the PDF if this Markdown changed. Evidence records the tested revision and the difference between implemented and production-verified. SA-031 is the final cross-document consistency check, not a reason to postpone normal documentation until Sprint 6.

13. Validation and KT sequence

The receipt records exact tests, source/path checks, PDF checks and limitations; the review records what was verified independently and what was sent back. Local validation uses Python 3.13 on Windows; production Docker uses Python 3.11 on Linux. Focused checks do not imply full-suite or platform-parity acceptance.

Maintainer commands, with required local dependencies installed:

```
python scripts/docs/run_kt_checks.py
python scripts/docs/build_kt_pdf.py
python scripts/docs/check_kt_docs.py
```

The test runner copies tracked source to ignored `analysis_data/`, excludes `.env/runtime` stores, disables `dotenv`, and blocks external transports and test subprocesses. It exercises existing tests, not SA-005's future clean CI gate or all independent financial invariants. The PDF builder needs Python `markdown2` and local `Playwright/Chromium`; it does not start the application.

KT session	Walkthrough
Product/data	Sections 1–4: a symbol, source dates, dimensions and research verdict.
Predictions/learning	Section 5: envelope row to feedback, weights and lessons; explain the grading gaps.
Portfolio/marksh eets	Sections 6–7: advice to transaction, the P/L example and different evaluation rules.
IPO/discovery	Section 8: calendar to captured facts, history, post-listing candidates, and the dark P3 path from deep dive to a stored, narrated and graded verdict.
Operations/road map	Sections 9–12: scheduled work to persisted output, outbox states and <code>last_error</code> , the dated email observation, and remaining PI changes.

Practical, non-code-intensive duties are in the separate Team Human Testing Guide. Testers should be able to explain outputs and compare evidence without reading prompts or reviewing code.